ADT LIST

Abstract Datatype List

2026 | Sabado

DEFINITION

Abstract Datatype

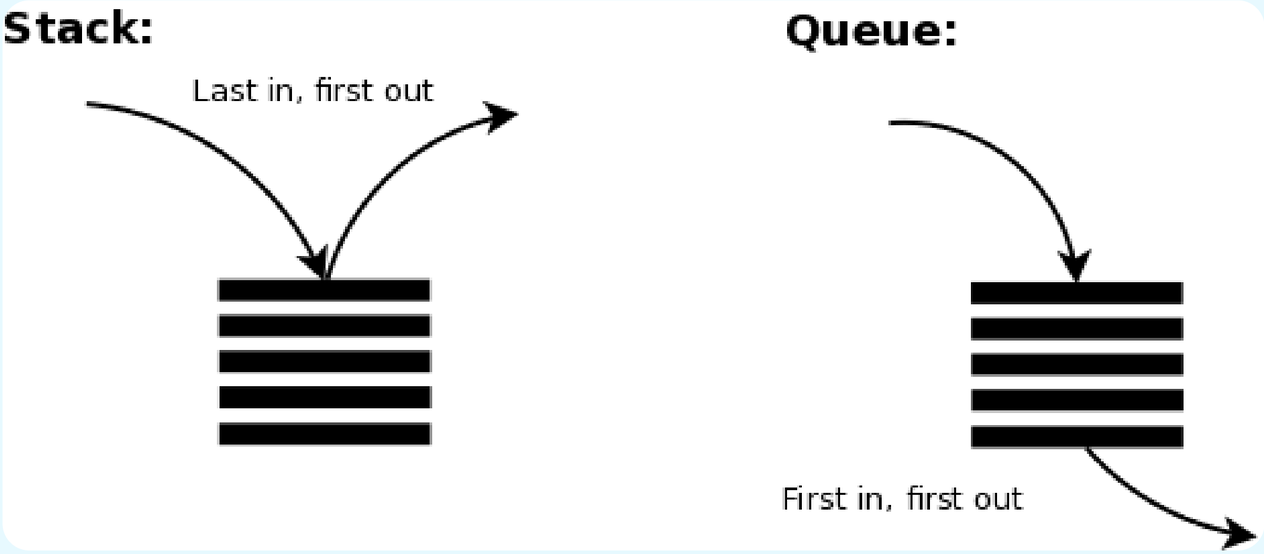
- is a conceptual model of a data structure that is defined by its behavior, or the set of operations that can be performed on it, rather than its concrete implementation.
- It separates the **WHAT** (the functionality) from the **HOW** (the implementation).

Encapsulation - Uses structs to hold the data and related functions to manipulate it.
Abstraction - The user only sees a simplified, high-level view of the data type.

DEFINITION

Abstract Datatypes

Ex. Lists, Stacks, Queue



ABSTRACT DATATYPES

Implementations

Array

Linked List

Cursor Based

Common Operations

- Insert
- Sort
- Delete/Remove
- Update
- Initialize
- Search/Locate
- Display/Retrieve
- Make Null/Empty

4 VARIATIONS OF LIST

Variation 1: LIST is a list that is static array and accessed by value

Variation 2: LIST is a list that is static array and accessed by pointer

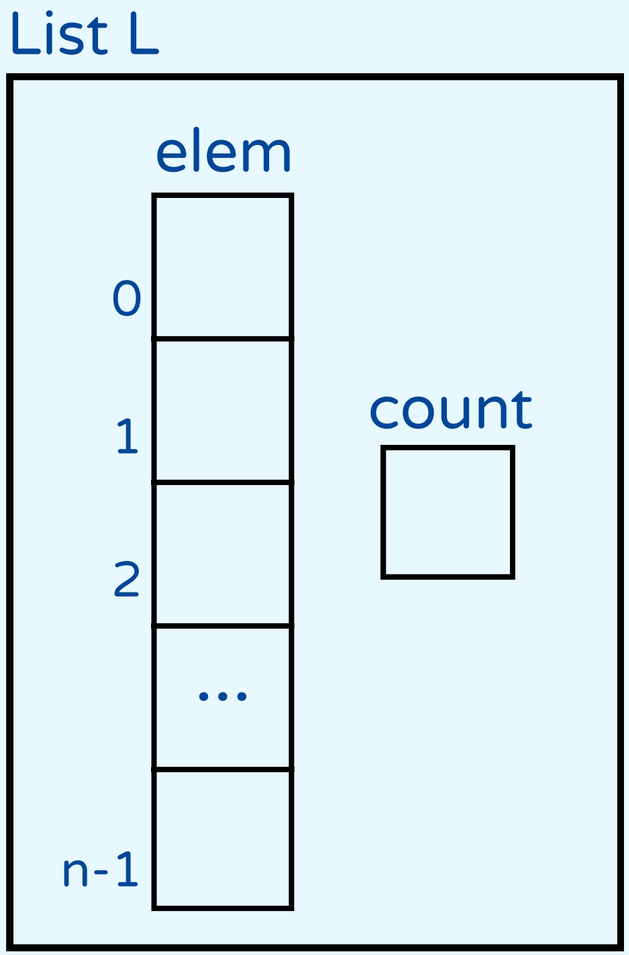
Variation 3: LIST is a list that is dynamic array and accessed by value

Variation 4: LIST is a list that is dynamic array and accessed by pointer

4 VARIATIONS OF LIST

	Memory Management	Structure Access	Scalability & Flexibility
Variation 1: Static, by Value	• Statically allocated at compile time • The size of the array is fixed and cannot be changed	• A copy of the entire list is made when passed to a function or assigned to a new variable	• Not scalable or flexible

VARIATION 1 LIST




```
#define MAX 10

typedef struct{
    int elem[MAX];
    int count;
} List;

List L;
```

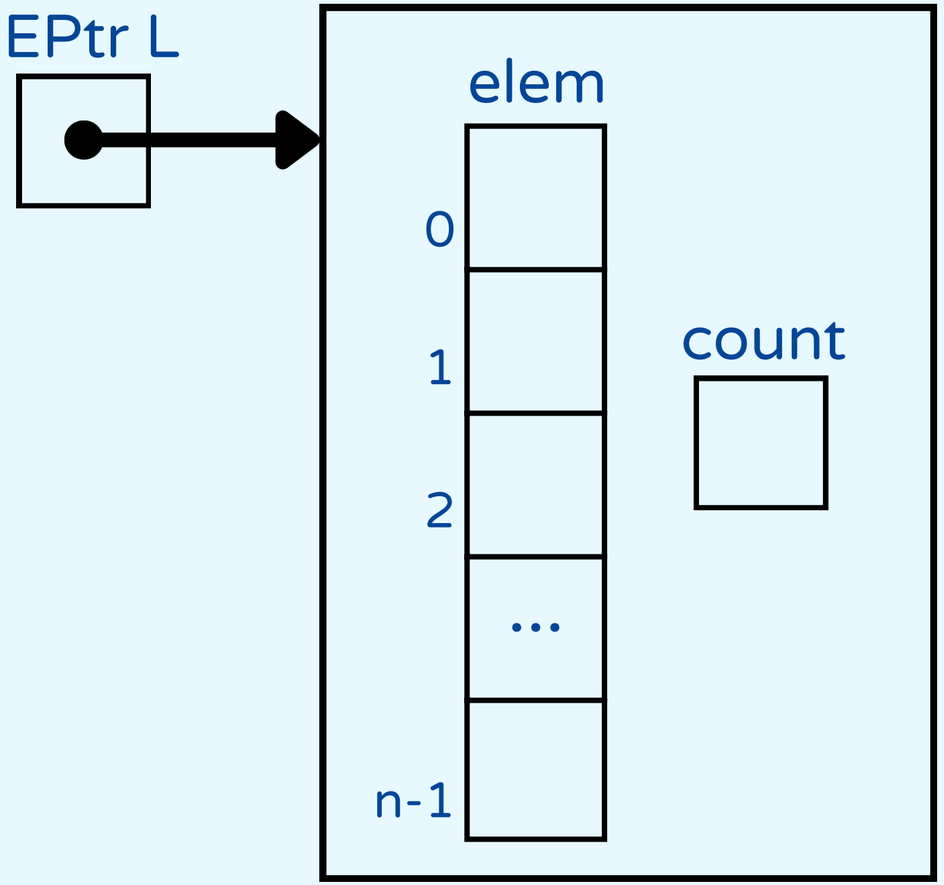
Function Prototypes:

- List initialize (List L);
- List insertPos (List L, int data, int position);
- List deletePos(List L, int position);
- int locate (List L, int data);
- List insertSorted (List L, int data);
- void display(List L);

4 VARIATIONS OF LIST

	Memory Management	Structure Access	Scalability & Flexibility
Variation 2: Static, by Pointer	• Statically allocated at compile time • The size of the array is fixed and cannot be changed	• Only the address is passed, not a full copy	• Not scalable or flexible

VARIATION 2 LIST




```
#define MAX 10
typedef struct{
    int elem[MAX];
    int count;
} Etype, *EPtr;

EPtr L;
```

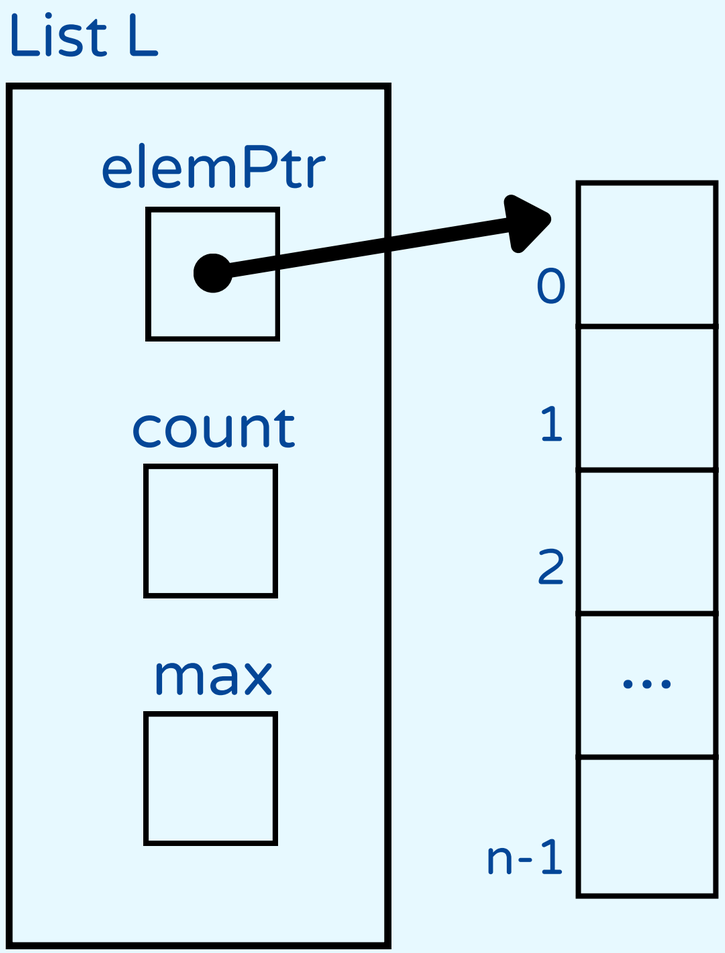
Function Prototypes:

- void initialize (EPtr L);
- void insertPos (EPtr L, int data, int position);
- void deletePos(EPtr L, int position);
- int locate (EPtr L, int data);
- int retrieve (EPtr L, int position);
- void insertSorted (EPtr L, int data);
- void makeNULL(EPtr L);

4 VARIATIONS OF LIST

	Memory Management	Structure Access	Scalability & Flexibility
Variation 3: Dynamic, by Value	• Dynamically allocated at run-time • The size can be changed as needed	• A copy of the entire list is made	• Scalable but not flexible • The list can grow or shrink in size. However, creating a new copy for every operation is inefficient

VARIATION 3 LIST




```
#define LENGTH 10
typedef struct{
    int  *elemPtr;
    int count;
    int max;
} List;
```

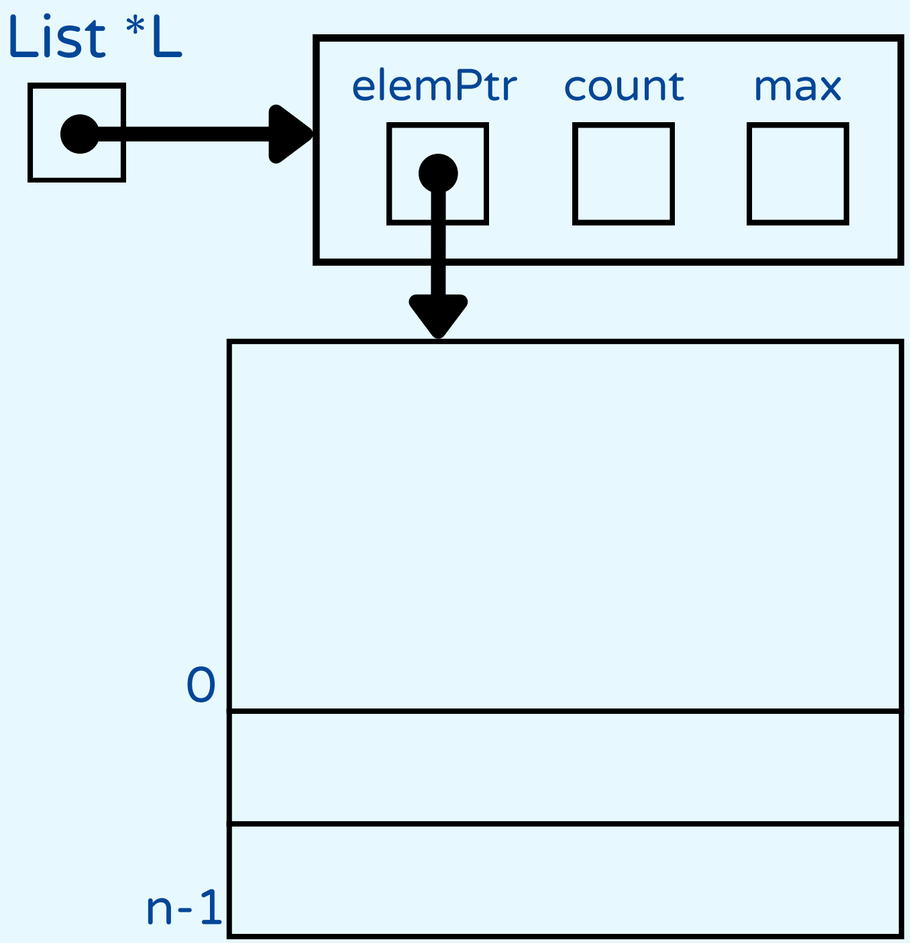
Function Prototypes:

- List initialize (List L);
- List insertPos (List L, int data, int position);
- List deletePos(List L, int position);
- int locate (List L, int data);
- List insertSorted (List L, int data);
- void display(List L);
- List resize(List L);

4 VARIATIONS OF LIST

	Memory Management	Structure Access	Scalability & Flexibility
Variation 4: Dynamic, by Pointer	• Dynamically allocated at run-time • The size can be changed as needed	• Only the address is passed, not a full copy	• Highly scalable and flexible • Since access is by pointer, only a small memory address is passed, making operations efficient

VARIATION 4 LIST




```
#define LENGTH 10
typedef struct{
    int *elemPtr;
    int count;
    int max;
} List;
```

Function Prototypes:

- void initialize (List *L);
- void insertPos (List *L, int data, int pos);
- void deletePos(List *L, int pos);
- void locate (List *L, int ID);
- studtype retrieve (List *L, int pos);
- void insertSorted (List *L, int data);
- void makeNULL(List *L);
- void resize(List *L);

STACK

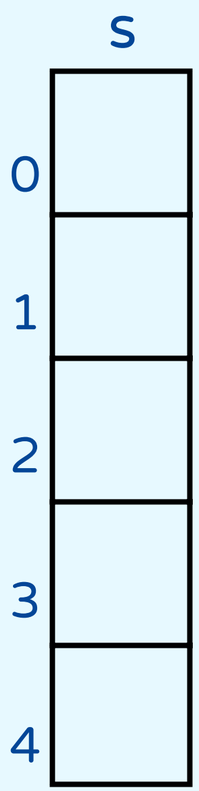
STACK

- special kind of list
- insertion and deletion takes place at one end (TOP)
- also known as pushdown

LAST IN, FIRST OUT

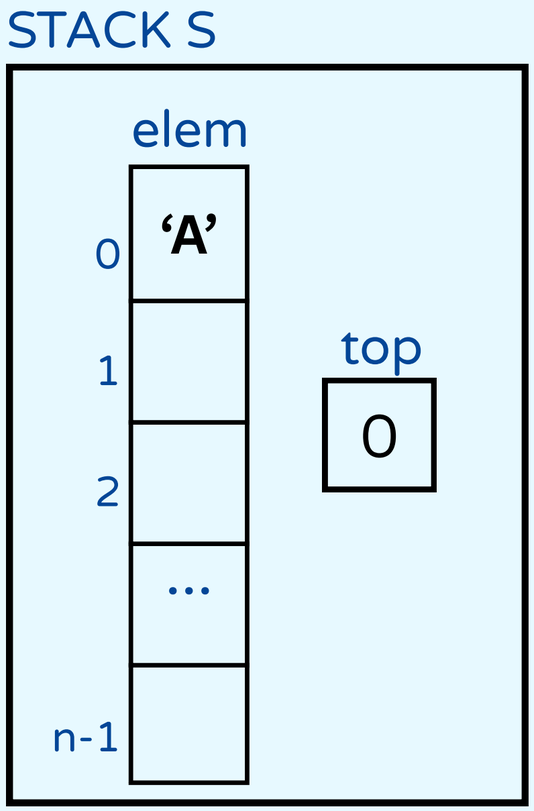
FIRST IN, LAST OUT

STACK OPERATIONS

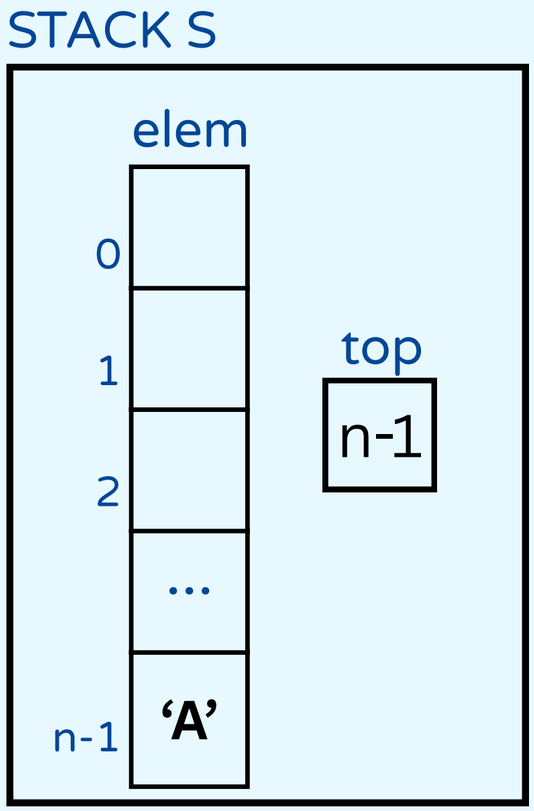


push(x,s[])	insert First
pop(s[])	deleteFirst
top(s[])	returns 1st element
peek(s[])	returns 1st element
isEmpty(s[])	checks if stack is empty
isFull(s[])	checks if stack is full
initStack(s[])	initialize garbage to empty
makeNULL(s[])	full to NULL
makeEmpty(s[])	full to empty

ARRAY IMPLEMENTATION

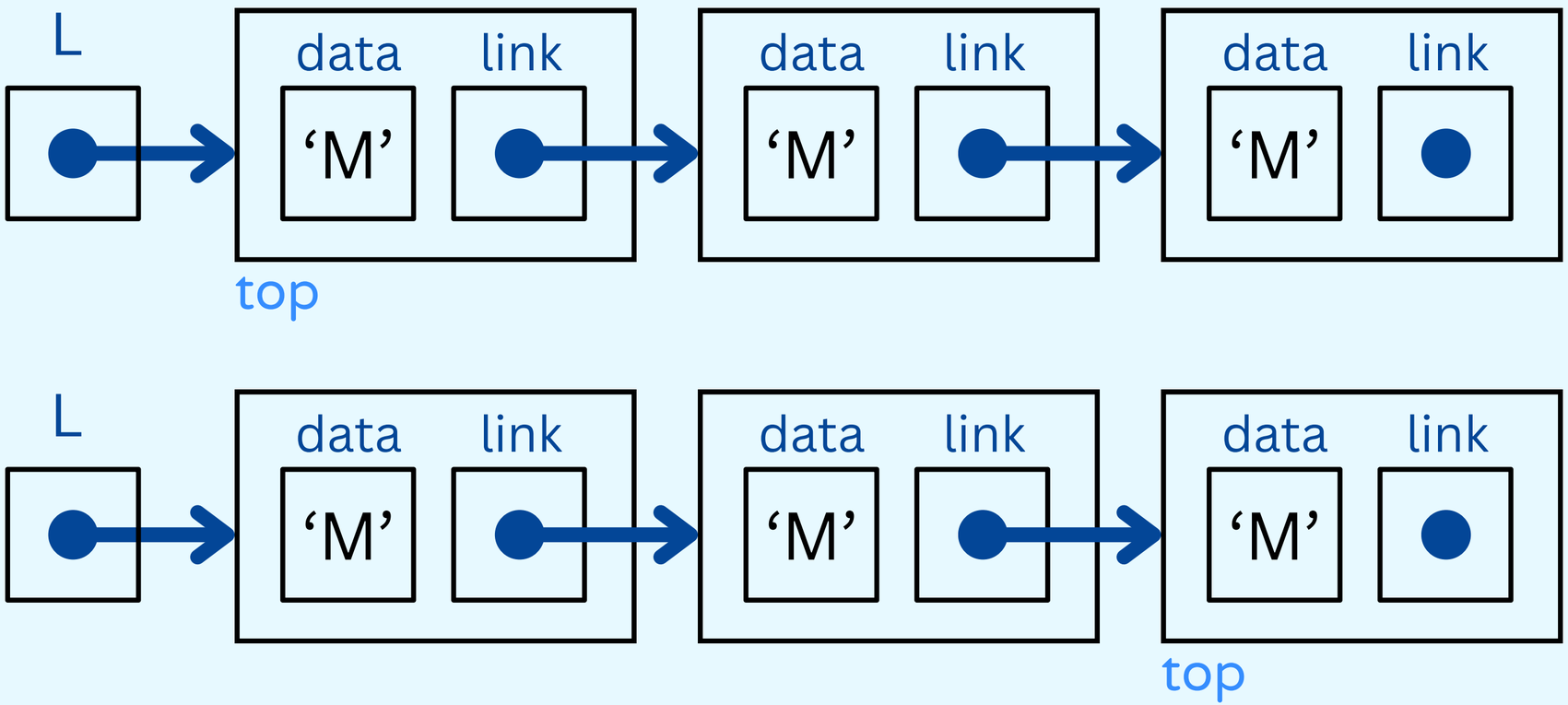


0 to n-1

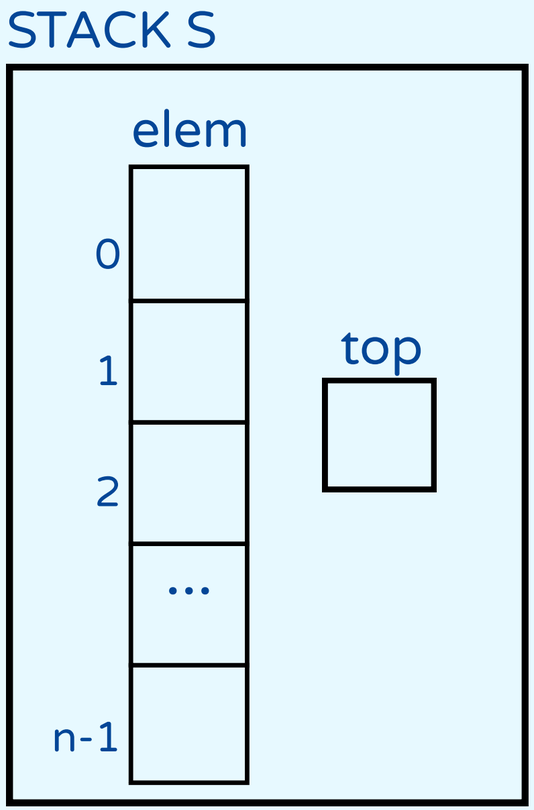


n-1 to 0

LINKED LIST IMPLEMENTATION

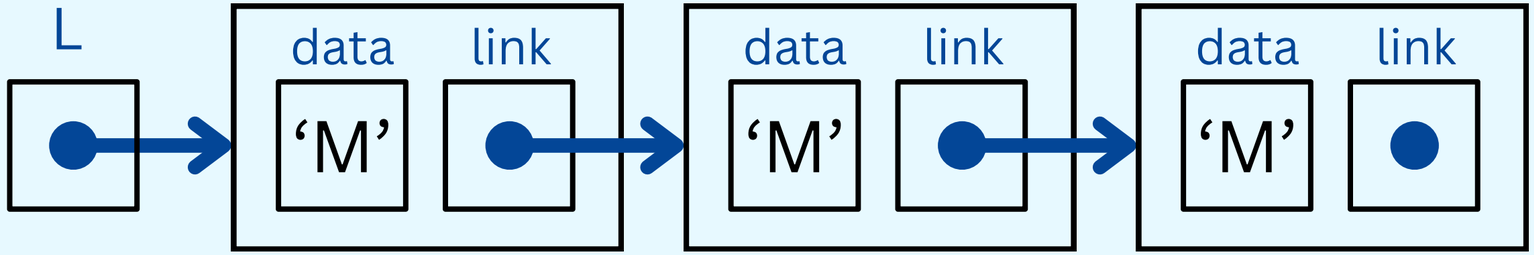


STACK O NOTATION



ADT LIST OPERATIONS	STACK OPERATIONS	
insertFirst	push	O(n)
deleteFirst	pop	O(n)
insertLast	push	O(1)
deleteLast	pop	O(1)

STACK O NOTATION



ADT LIST OPERATIONS	STACK OPERATIONS	
insertFirst	push	O(1)
deleteFirst	pop	O(1)
insertLast	push	O(n)
deleteLast	pop	O(n)

QUEUE

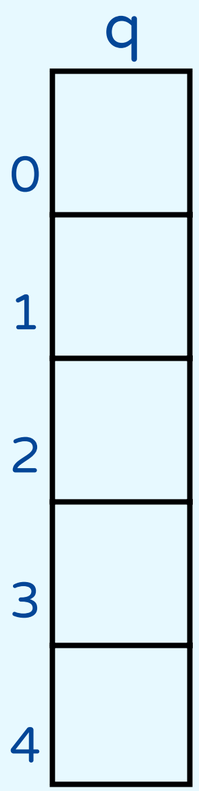
QUEUE

- special kind of list
- does operations on both ends
    - 2 endpoints
- insertion is done on one side, deletion is done on the other (REAR/FRONT)

FIRST IN, FIRST OUT

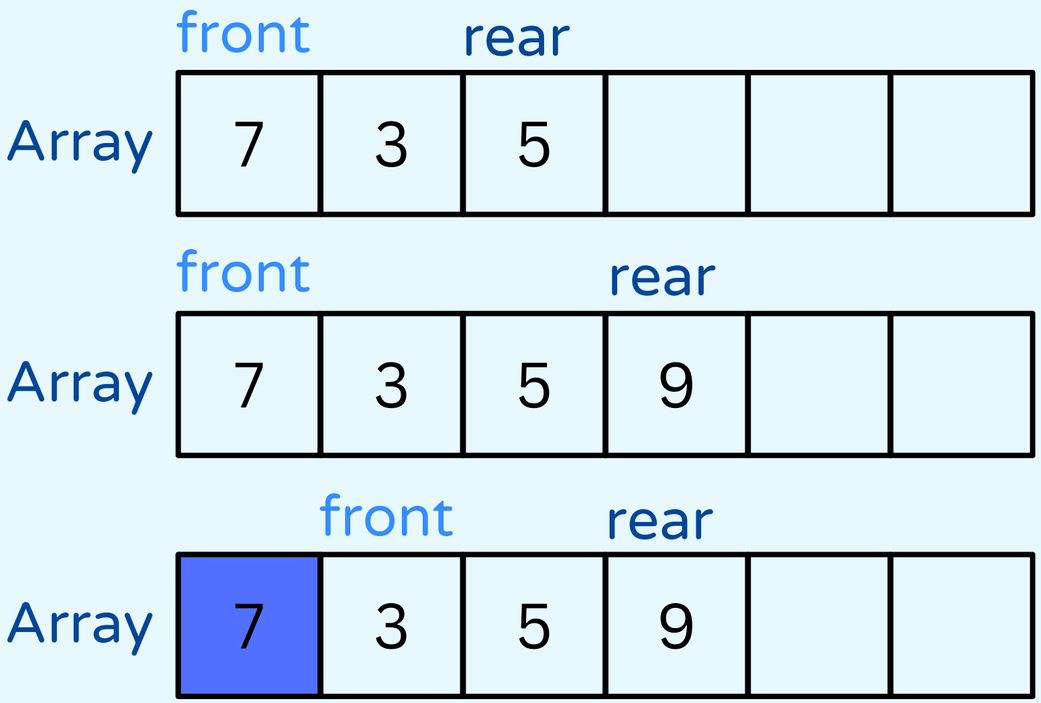
LAST IN, LAST OUT

QUEUE OPERATIONS



enqueue(x,q[])	insert rear
dequeue(q[])	delete front
front(q[])	returns 1st element
isEmpty(q[])	checks if queue is empty
isFull(q[])	checks if queue is full
initQueue(q[])	initialize garbage to empty
makeNULL(q[])	full to NULL
makeEmpty(q[])	full to empty

ARRAY IMPLEMENTATION



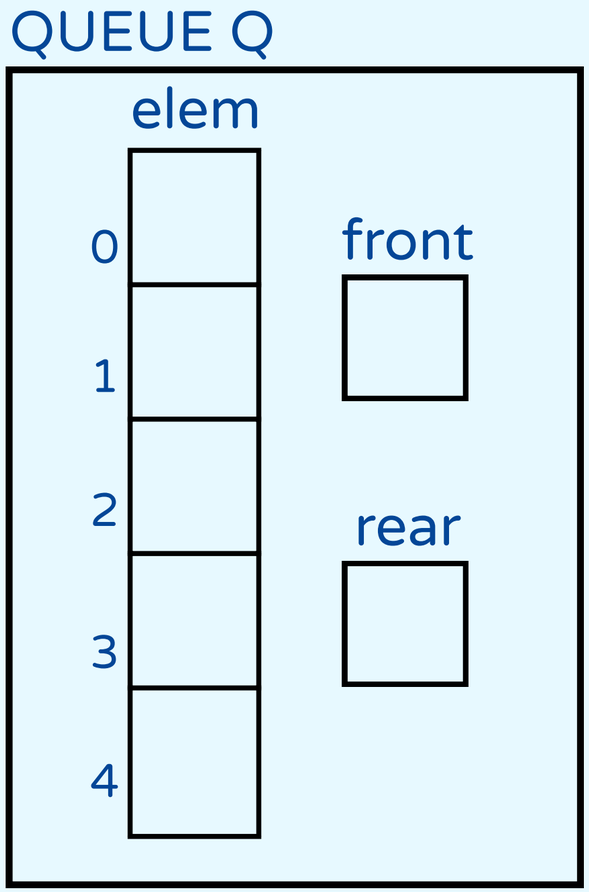
`enqueue(9)`

`dequeue()`

Problem:

- can't access the first index
- array space for values become limited

ARRAY IMPLEMENTATION



Empty
- front = -1
- rear = -1

Full
- front = 0
- rear = MAX-1

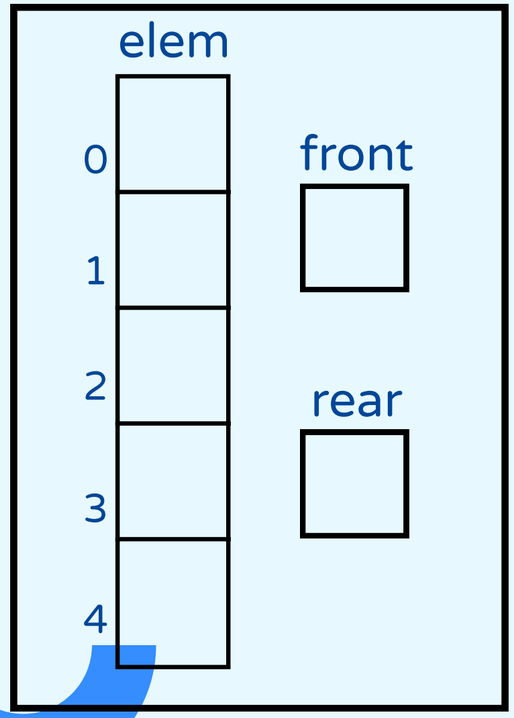
front and rear
- index of array

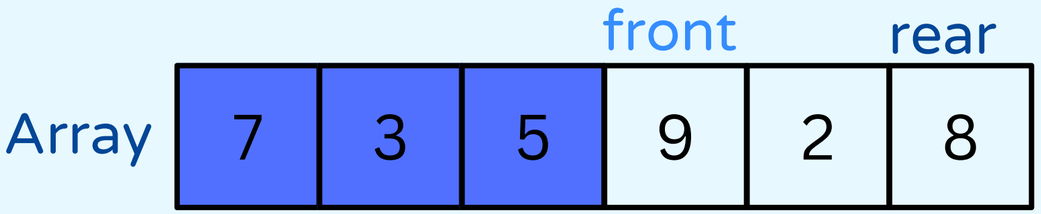
ARRAY IMPLEMENTATION

Problem:

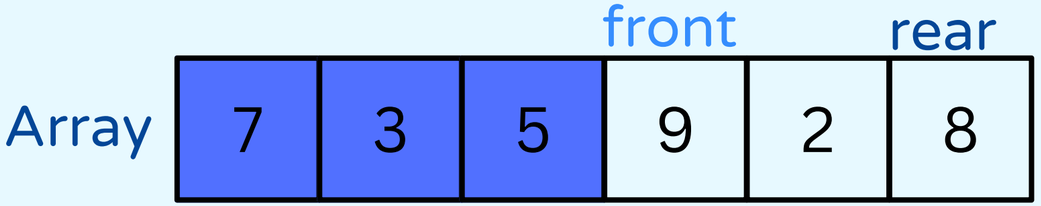
- run out of space

QUEUE Q



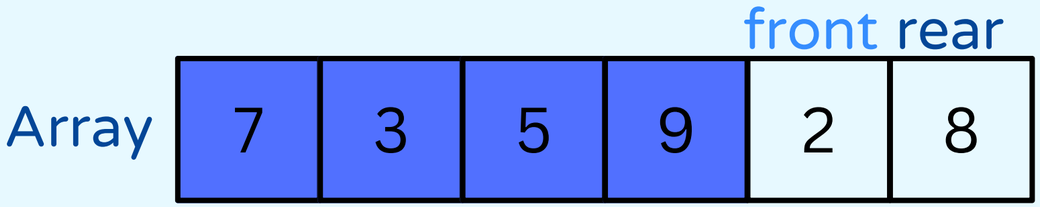


`enqueue(8)`

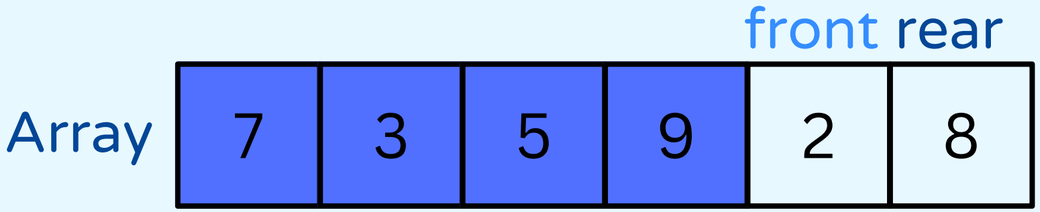


`enqueue(1)`

`error! full!`



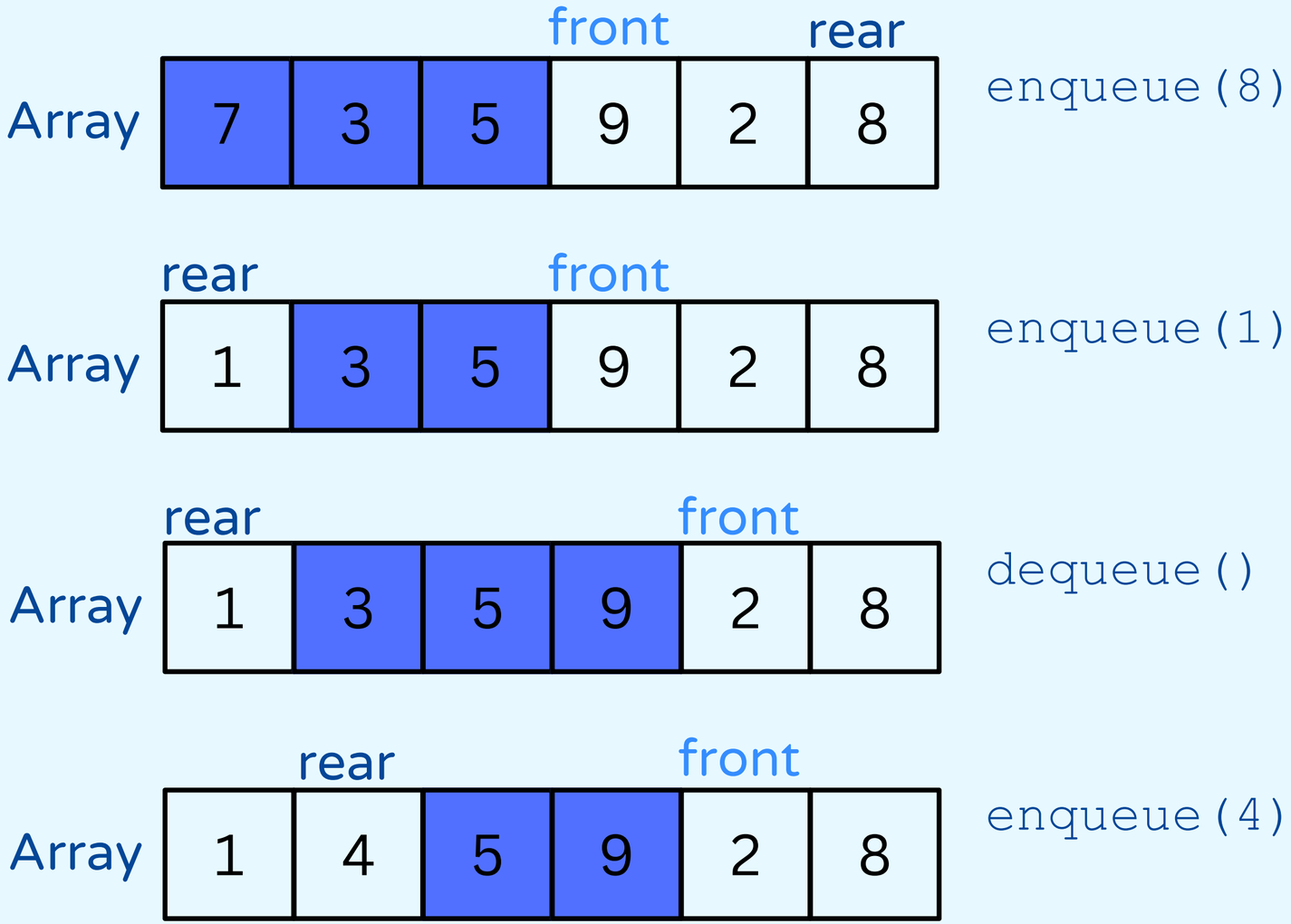
`dequeue()`



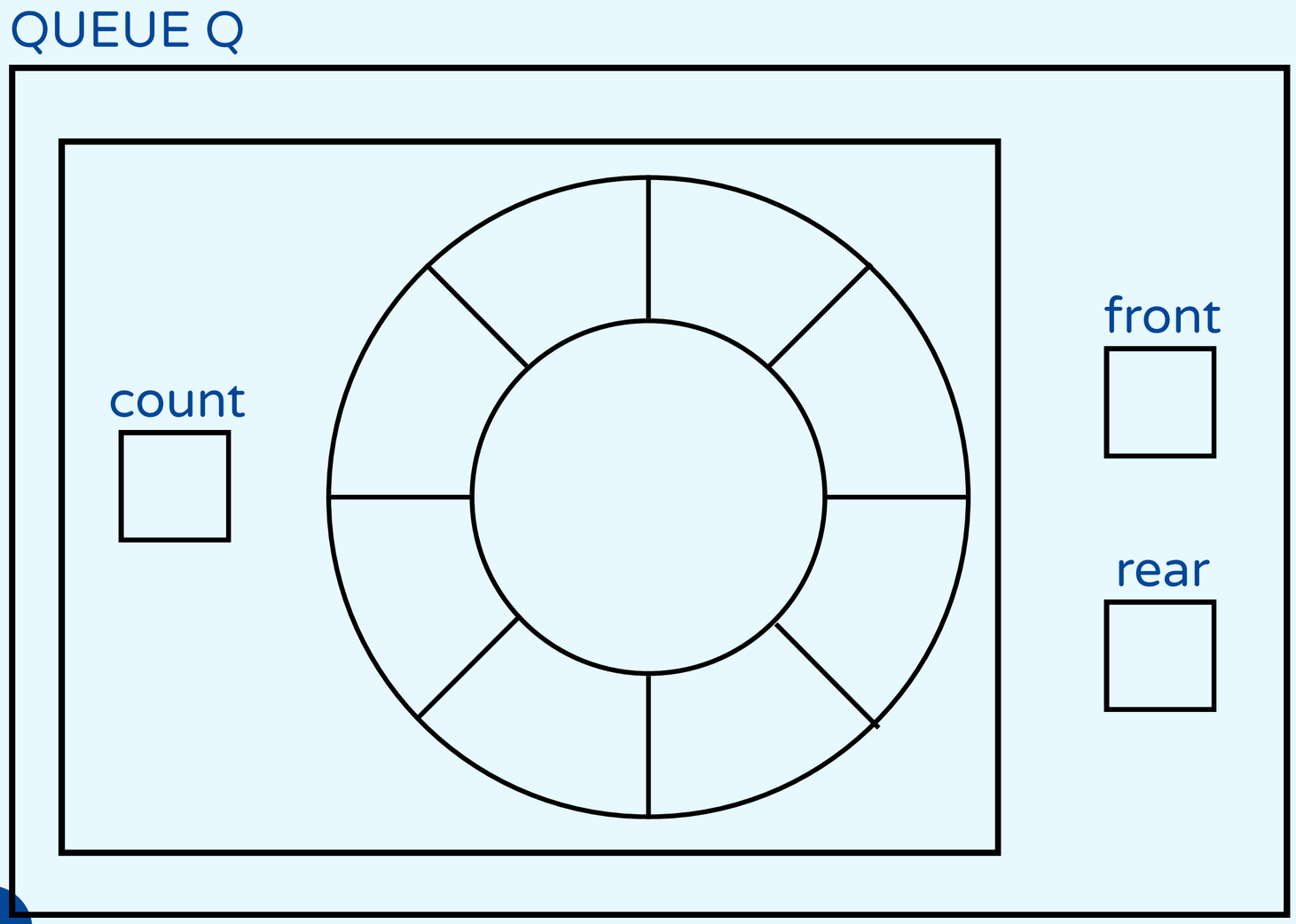
`enqueue(4)`

`error! full!`

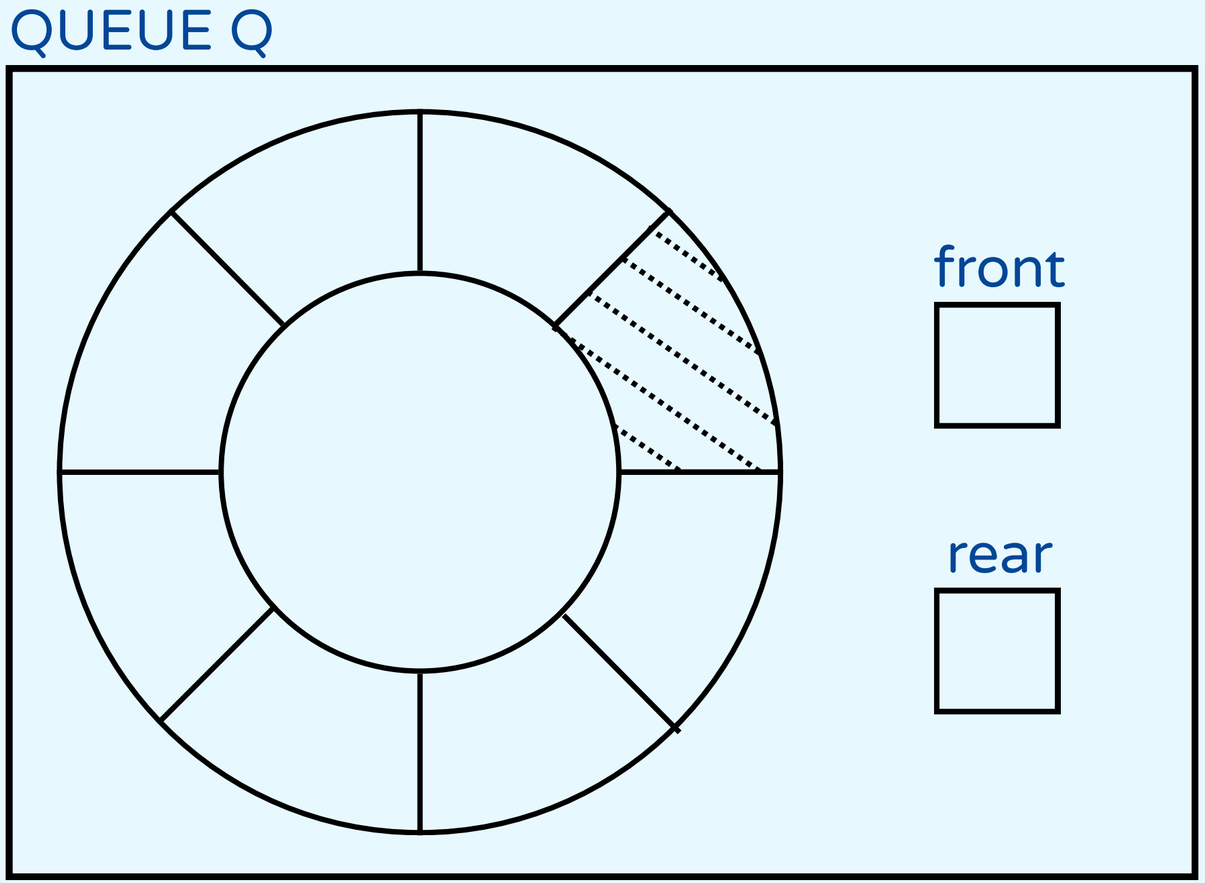
SOLUTION: Circular Array



SOLUTION: Circular Array (V1)

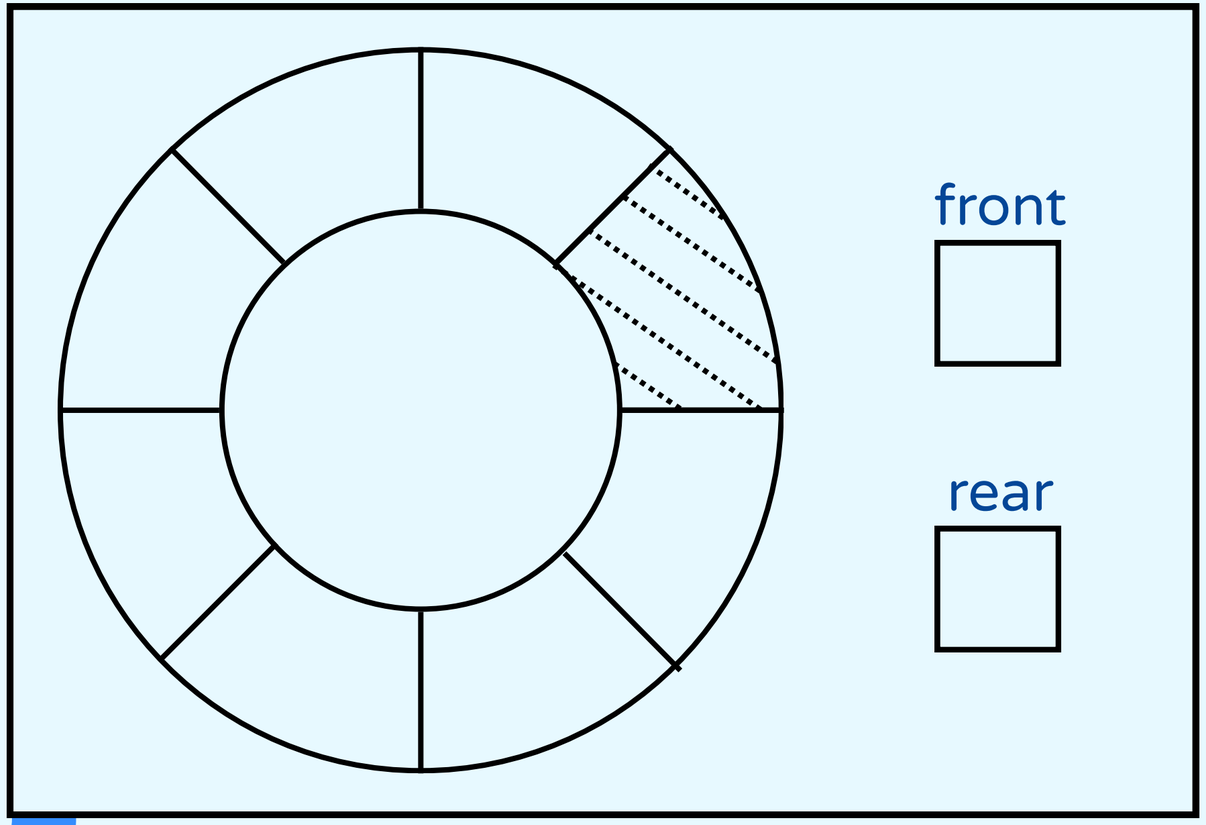


SOLUTION: Circular Array (V2)



CIRCULAR ARRAY

QUEUE Q



Empty

front == rear + 1

rear == front - 1

Full

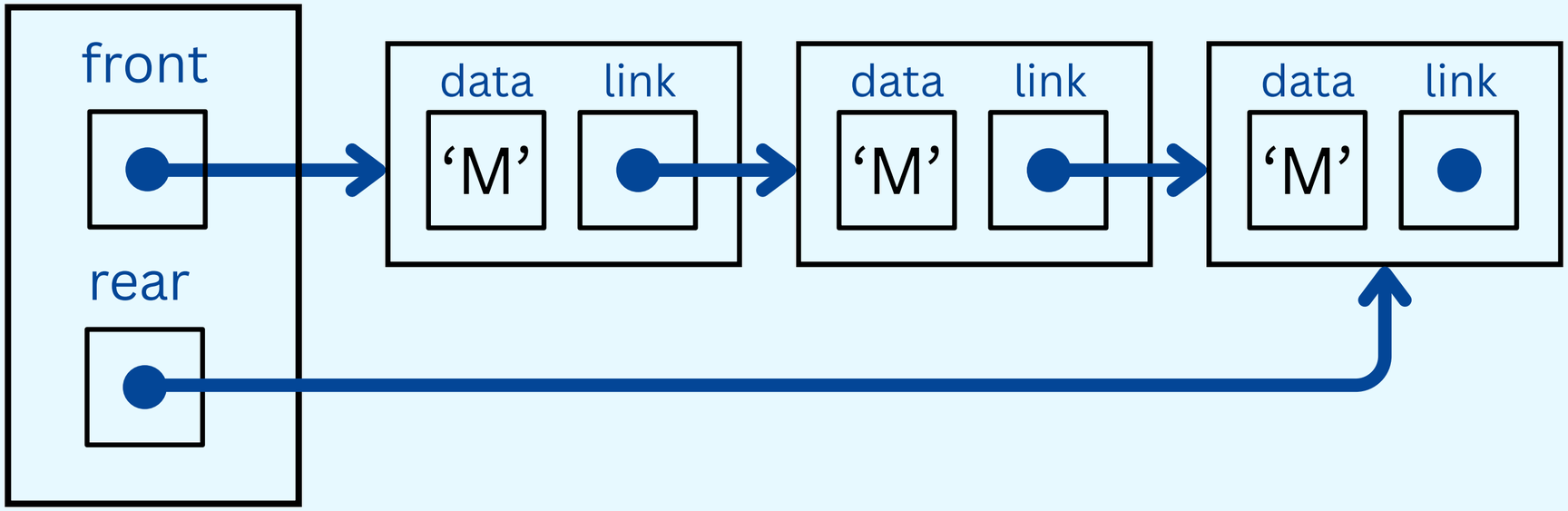
front == rear + 2

rear == front + 2

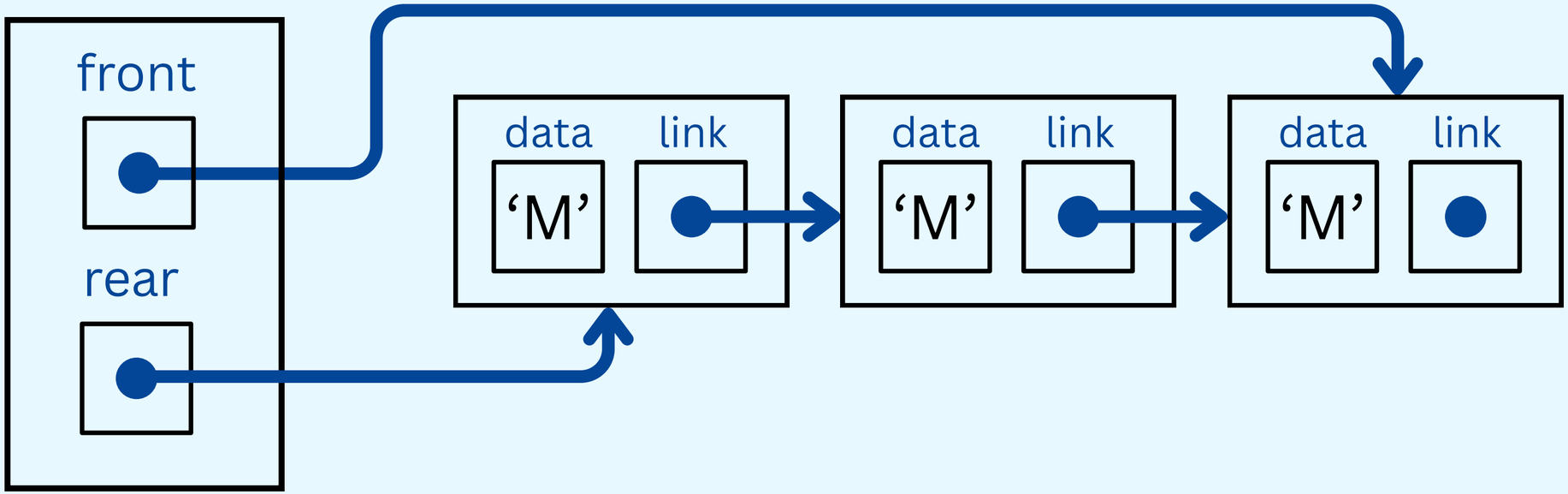
1 element

front == rear

LINKED LIST IMPLEMENTATION



LINKED LIST IMPLEMENTATION



OPERATIONS

	ADT List	Stack	Queue
Access	*like usual* **array:** List[x].data **linked list:** List->data `retrieve( key );`	*from one endpoint* `top(S); peek(S);`	*from the front endpoint* `front(Q);`
Insert	First, Last, Position, Sorted	`push(S,data);`	`enqueue(Q,data);`
Deletion	First, Last, Position, Key	`pop(S);`	`dequeue(Q);`

Traversal	`int x;`
	`for(x; x<List.count; x++)`
	`{}`
		`while(!isEmpty(S)){`
		`  push(S,data);`
		`  pop(S);`
		`}`
			`while(!isEmpty(Q)){`
			`  enqueue(Q,data);`
			`  dequeue(Q);`
			`}`

THANK YOU